Ex. no 6

Practical-6

22/9/26

Error Detection and Correction

Aim

Write a program to implement error detection and correction using HAMMING code concept. Make a test run to input data stream and verify error correction feature

Sender code :-

program features: 1. Input to sender file should be a text
2. Apply hamming code concept on binary data
3. Save this output 4. Create a reciever program with features

Coder :

```
def calculate_parity_bits(data_bits):
    r=0
    while (2**r) < (data_bits+r+1):
        r+=1
    return r
def hamming_encode(data):
    m= len(data)
    r= calculate_parity_bits(m)
    totalbits = m+r
    hamming= [0]* (total_bits+1)
    data_index=0
    for position in range(1, totalbits+1):
        if position & (position-1):
            hamming[position] = int(data(data_index))
            data_index+=1

    parity_position=1
    while parity_position <= total_bits:
        parity=0
    for position in range(1, total_bits+1):
        if position & parity_position:
```

```
            parity_n = hamming_position
        encoded = ""
        for position in range(1, total_bits+1):
            encoded += str(hamming[position])
        return encoded
    with open("input.txt", "r") as file:
        text = file.read()
        print("Original text")
        print(text)
    with open("channel.txt", "w") as channel:
        for character in text:
            channel.write(encoded + " ")
    print("\nHamming encoded data saved to channel.txt")
```

Reciever program:

Features
1. Reciever program should recieve the input
2. Apply hamming code on binary data
3. If there is error, display the position of the error

Code:

```
def hamming_decode(block):
    total_bits = len(block)
    hamming_code = [0]
for bit in block:
    hamming.append(int(bit))
    error_position = 0
    parity_position = 1
while parity_position <= total_bits:
    parity=0
for position in range(1, total_bits+1):
    if position & parity_position:
```

```
        Parity = hamming [position]
with open ('channel text', 'r') as channel :
    print ("Recieved data")
    result = " "
    for block in data :
        character = hamming_decode (block)
        result += character
        print("\n Original message after correcting
print(result)
```

Result :

The program has been executed successfully